A CORE THESIS

EATP

Enterprise Agent Trust Protocol

Why Trust Lineage Is the Missing Infrastructure for Enterprise AI

AUTHOR Dr. Jack Hong, Singapore Management University

VERSION 2.2 | March 2026 (reference-implementation note revised May 2026)

SERIES Terrene Foundation White Paper

EATP: A Core Thesis

Enterprise Agent Trust Protocol

Dr. Jack Hong, Singapore Management University · Version 2.2 | March 2026 (reference-implementation note revised May 2026) · CC BY 4.0

Abstract

The Enterprise Agent Trust Protocol (EATP) addresses a structural gap in enterprise AI governance: the absence of a systematic, verifiable mechanism for tracing autonomous AI actions back to human authority. This paper presents the protocol's core insight — separating the moment of trust establishment from the moment of trust verification — and describes five cryptographically linked elements (Genesis Record, Delegation Record, Constraint Envelope, Capability Attestation, Audit Anchor) that form an unforgeable (under standard cryptographic assumptions: unforgeability of Ed25519 digital signatures and collision resistance of SHA-256) Trust Lineage Chain.

Version 2.2 extends the protocol with optional structured reasoning traces — machine-verifiable records of WHY a trust decision was made, complementing the existing WHO/WHAT/WHEN. Reasoning traces attach to Delegation Records and Audit Anchors with enterprise confidentiality classification (PUBLIC through TOP_SECRET), dual-binding cryptographic signing (reasoning trace hash bound into the parent record's signature, with independent reasoning signature for content verification), and a privacy model based on SD-JWT selective disclosure.

The reference implementation — the `kailash.trust` namespace of **kailash-py** (v2.28.1, Apache 2.0) — implements the full protocol: four operations (ESTABLISH, DELEGATE, VERIFY, AUDIT), five constraint dimensions, structured reasoning traces with confidentiality classification, inter-agent messaging with replay protection, pluggable persistent storage (in-memory, filesystem, PostgreSQL), an MCP server for direct integration with AI agent frameworks, a CLI for trust-lifecycle management, and interoperability with six industry-standard credential formats (JWT, W3C Verifiable Credentials, DID, UCAN, SD-JWT, Biscuit).

The specification is published under CC BY 4.0. All reference implementations are released under the Apache 2.0 open-source license. EATP is a public good maintained by the Terrene Foundation.

Disclosure

I developed EATP and built the Kailash reference implementation. The author is the designer, builder, sole operator, and evaluator of the system described—this dual role creates an inherent conflict of interest that readers should weigh when assessing claims. No independent party has validated the claims in this paper. The protocol has not undergone formal verification or academic peer review. Security properties are argued informally, not proven.

The EATP specification is published under CC BY 4.0 (Creative Commons), meaning anyone can read, share, and build upon it. The reference implementation is released under the Apache 2.0 open-source license, meaning the source code is publicly available, and anyone is legally entitled to use it, modify it, and build commercial products with it, permanently and irrevocably.

Two patent applications are pending (PCT/SG2024/050503 and P251088SG; favorable IPRP received, national phase filings in progress in Singapore and the United States; no patent has been granted). The Apache 2.0 license includes an automatic patent grant (Section 3 of the Apache License) providing users a perpetual, royalty-free patent license for the contributed code, with defensive termination if the user initiates patent litigation. The specification is CC BY 4.0. Both licenses are irrevocable under their respective terms. The author is the founder of the Terrene Foundation, which publishes the specification and maintains the reference SDK. No external funding was received for this work.

The Accountability Gap

A purchasing agent approves vendor invoices. Under traditional governance, a human reviews each invoice, applies judgment, and signs off. Accountability is clear: the approving human answers for the decision.

Now give that function to an AI agent. It processes invoices at machine speed: hundreds per hour instead of dozens per day. Significant operational value. But when an improper payment goes through, a question arises that nobody can answer satisfactorily:

"The AI approved it" is not an answer.

It does not satisfy the board asking which executive bears responsibility. It does not satisfy the regulator asking who violated the compliance requirement. It does not satisfy the auditor asking where the decision trail is. It does not satisfy the court asking who can be held liable. And it certainly does not satisfy the investigator asking why that particular agent was authorized to act in the first place.

This is not a hypothetical concern. Organizations are deploying AI agents today, and the accountability question is being deferred, not solved. Every deferred question accumulates institutional risk. And that risk is growing at the same rate as AI adoption.

The question this paper addresses is not "how do we build better AI." It is "how do we institutionalize the accountability structures that make enterprise AI trustworthy." Doing better things, not doing things better.

Why Traditional Approaches Fail

Four approaches dominate current thinking. None of them work.

Human-in-the-loop requires a human to approve every AI action. This preserves accountability but eliminates the economic value of automation. If a human must approve each invoice, you have built an expensive validation layer, not an autonomous agent.

Rule-based systems attempt to pre-specify every correct action. This works for simple, static domains. It fails for complex, dynamic environments where rules cannot anticipate every situation. The rule system becomes as complex as the problem it governs.

Post-hoc review allows AI to act autonomously with humans reviewing afterward. This captures the speed advantage but cannot undo harm. By the time review catches the problem, the payment has been made, the email sent, the decision executed.

Trust by default assumes AI can be trusted to act correctly. This ignores that AI systems make errors, can be manipulated, and operate without the contextual judgment that humans bring to decisions.

Each approach makes the same mistake: it treats the accountability question as a byproduct of implementation rather than as a design requirement. None of them provides a systematic, verifiable answer to "who is accountable for what this AI agent just did?"

The Core Insight

The problem becomes tractable once you recognize that it conflates two distinct moments:

Trust establishment: The decision that an agent should be permitted to act within certain boundaries. This requires human judgment: understanding context, evaluating risk, accepting responsibility.

Trust verification: The check that a specific action falls within those boundaries. This is a mechanical comparison that can be performed by a machine in milliseconds.

Traditional governance performs both moments together: a human evaluates each action against their judgment and approves or rejects. This works when decisions are few and slow. It fails when decisions are many and fast.

EATP separates these moments. Humans invest judgment once when establishing trust: defining who can act, within what boundaries, under whose authority. The system verifies continuously whether specific actions fall within established boundaries.

Separate the moment of trust establishment from the moment of trust verification. Humans define boundaries thoughtfully. Machines enforce them continuously. Every action traces back to human decisions through verifiable chains.

This is EATP's contribution. Not a new theory of AI governance. A specific architectural separation that preserves human accountability while enabling autonomous operation.

The Five Elements: The Trust Lineage Chain

EATP defines five cryptographically linked elements. Together they form the Trust Lineage Chain: an unbroken, verifiable connection from any AI action back to human authority.

1. Genesis Record

The organizational root of trust.

A human executive cryptographically commits: "I accept accountability for this AI governance framework." This is not a technical act. It is a governance act, comparable to signing a corporate charter or accepting regulatory obligations.

No AI creates its own genesis record. Trust originates in human commitment.

The Genesis Record establishes who is ultimately accountable. It binds a specific human authority, a CEO, a board, a governance committee, to the entire downstream chain of AI operations. Everything that follows traces back to this commitment.

2. Delegation Record

Authority transfer with constraint tightening.

The critical rule: delegations can only reduce authority, never expand it. A manager with $50,000 spending authority can delegate $10,000 to an AI agent. They cannot delegate $75,000. The protocol rejects delegations that exceed delegator authority at creation time.

This mirrors how healthy human organizations actually work. Authority flows downward and narrows. A CFO delegates budget authority to department heads, who delegate smaller budgets to team leads. No one in the chain creates authority they do not possess. EATP makes this organizational principle cryptographically enforceable.

Optionally, a Delegation Record may include a **reasoning trace** — a structured record of why the delegation was made, what alternatives were considered, and what evidence supported the decision. This answers a question that traditional access control never asks: not just "who delegated what to whom," but "why did they decide to do so?" Reasoning traces have their own confidentiality classification and a dual-binding cryptographic model: the reasoning trace hash is bound into the parent record's signature (preventing post-hoc substitution), while a separate reasoning signature enables independent verification of the reasoning content. This allows the rationale to be verified, redacted, or withheld independently of the delegation itself.

3. Constraint Envelope

Multi-dimensional operating boundaries across five dimensions:

Financial: Transaction limits, spending caps, cumulative budgets.

Operational: Permitted and blocked actions. What the agent may do and what it may not.

Temporal: Operating hours, blackout periods, time-bounded authorizations.

Data Access: Read and write permissions, PII handling rules, data classification boundaries.

Communication: Permitted channels, approved recipients, tone and content guidelines.

Constraint Envelopes encode human judgment as machine-verifiable boundaries. The judgment is: "Given what I know about this role, these risks, and these requirements, this agent should operate within these limits without additional approval." That judgment is human. The verification is machine.

The choice of five dimensions is a design decision. These dimensions emerged from analysis of common organizational constraints. Other decompositions may be equally valid. But these five cover the most frequent governance concerns enterprises face.

4. Capability Attestation

A signed declaration of what an agent is authorized to do.

This solves a problem that governance frameworks rarely address: capability drift. Over time, AI agents gradually take on tasks they were never explicitly authorized to perform. A report-generating agent starts making data corrections. A scheduling agent begins sending external communications. Each incremental expansion seems minor. Cumulatively, the agent operates far outside its intended scope.

Capability Attestation makes authorized scope explicit and verifiable. If the agent attempts an action outside its attested capabilities, the system catches it, not after the fact, but before execution.

5. Audit Anchor

A permanent, tamper-evident execution record.

Each Audit Anchor hashes the previous anchor. Modifying any record invalidates the chain from that point forward, making tampering detectable.

Honest limitation: A simple linear hash chain has known weaknesses. An attacker who compromises the system can modify all records from the point of compromise forward while maintaining hash consistency. Production implementations should use stronger mechanisms, such as Merkle trees or periodic external checkpointing, after independent security review.

The Audit Anchor answers the questions that matter in an investigation: What happened? Who authorized it? What constraints applied? Were those constraints respected? And optionally, why did the agent choose this action? An Audit Anchor may include a **reasoning trace** — the agent's structured rationale for its decision, with the same confidentiality classification and dual-binding signing model as delegation reasoning traces. These answers are cryptographically linked to the evidence, not reconstructed from memory or assertion.

How Verification Works

When an agent proposes an action, the system checks the entire Trust Lineage Chain. But verification is not binary. EATP defines a gradient:

Result	Meaning	Action
Auto-approved	Within all constraints	Execute and log
Flagged	Near constraint boundary	Execute and highlight for review
Held	Soft limit exceeded	Queue for human approval
Blocked	Hard limit violated	Reject with explanation

This gradient focuses human attention where it matters: near boundaries and at limits, rather than distributing it across all actions. A manager does not need to see the routine transactions. They need to see the ones approaching limits or entering grey areas.

The reference implementation provides three verification levels that trade off speed for thoroughness: QUICK (~1ms, hash and expiration check), STANDARD (~5ms, capability and constraint validation), and FULL (~50ms, cryptographic signature verification of the entire chain including reasoning trace hash and signature verification when present). These latency figures are design targets, not benchmarks; actual performance depends on implementation, storage backend, and chain depth. A `REASONING_REQUIRED` constraint mandates that reasoning traces be present. The name expresses organizational policy intent; enforcement escalates with the verification gradient: at STANDARD level, missing reasoning produces a non-blocking warning; at FULL level, missing reasoning when `REASONING_REQUIRED` is active causes verification failure. When reasoning traces are present at FULL level, both hash integrity and signature validity are verified. Production deployments can select the appropriate level based on the sensitivity of each operation.

STANDARD verification risk. STANDARD verification omits cryptographic signature checks. It provides no cryptographic defense — its security relies entirely on trust store integrity. In distributed deployments where trust stores are replicated or cached, an attacker with write access to a local replica could insert fabricated records that STANDARD accepts. Three mitigations apply: (1) the trust store is protected by OS-level or database access controls, limiting who can write to it; (2) FULL verification is always available and should be used for high-consequence actions; (3) Audit Anchors provide post-hoc detection — even if a fabricated record is accepted at STANDARD level, the resulting audit trail creates evidence for subsequent investigation.

Five Trust Postures

EATP supports graduated autonomy through five trust postures:

Posture	Autonomy	Human Role
Pseudo-Agent*	None	Human in-the-loop; agent is interface only
Supervised	Low	Human in-the-loop; agent proposes, human approves
Shared Planning	Medium	Human on-the-loop; human and agent co-plan
Continuous Insight	High	Human on-the-loop; agent executes, human monitors
Delegated	Full	Human over-the-loop; remote monitoring

*"Pseudo-Agent" denotes that the software acts as an interface (not an autonomous agent); the human performs all reasoning and decision-making.

Postures can be upgraded as trust builds through demonstrated performance. They can be downgraded instantly if conditions change. This is how organizations actually manage delegation: earn trust through track record, lose it through failure.

Cascade Revocation

When trust is revoked at any level, all downstream delegations are invalidated within a bounded window Δ determined by credential validity periods and propagation latency, after which no further actions by downstream agents are authorized. No orphaned agents continue operating after their authority source has been removed and the propagation window has closed.

Important caveat: "Immediate and atomic" is an architectural goal, not a guaranteed property. Distributed systems have propagation latency. Cached credentials may persist briefly. During the propagation window Δ, a revoked agent may continue operating with cached credentials.

Mitigations: short-lived credentials with five-minute validity windows bound Δ. Push-based revocation notification accelerates propagation. Action idempotency prevents double-execution during reconnection. Revocation is permanent and irrevocable in the trust store. If a downstream agent is offline when revocation occurs, its next verification attempt fails. Organizations should design for the reality that propagation is not truly instantaneous and build compensating controls for the gap.

Prior Art: What EATP Builds On

EATP does not emerge from nothing. An honest assessment of prior art is essential.

Control plane / data plane separation comes from software-defined networking and Kubernetes. The architectural pattern of separating policy decisions from policy enforcement is well established.

PDP/PEP architecture (Policy Decision Point / Policy Enforcement Point) from XACML provides the pattern of centralized policy decisions with distributed enforcement.

OAuth 2.0 scopes (RFC 6749) demonstrate delegated authorization with constraint tightening, the same principle that EATP applies to AI agent delegation.

SPIFFE/SPIRE provides workload identity and trust bootstrapping for distributed systems.

PKI certificate chains established the pattern of hierarchical trust with cryptographic verification.

Haber & Stornetta (1991) developed timestamping mechanisms for digital documents. **Merkle (1980)** formalized hash trees for efficient verification.

Irving et al. (2018) at DeepMind proposed AI safety via debate, a framework where AI agents argue for and against proposed actions before a human judge. EATP's verification gradient addresses a related but distinct problem: rather than having agents debate each action, EATP pre-establishes the boundaries within which actions are acceptable and verifies compliance mechanically. The two approaches are complementary; debate mechanisms could serve as a human-review process for actions that EATP holds for approval.

NIST AI Risk Management Framework (2023) recommends governance structures that include explicit authorization chains, documented risk boundaries, continuous monitoring, and auditable decision records. At a high level, EATP's five elements map to these recommendations: Genesis Records establish governance authority, Constraint Envelopes encode risk boundaries, trust postures implement graduated monitoring, and Audit Anchors provide decision records. NIST does not prescribe a specific protocol; EATP is one concrete implementation of the structural approach NIST describes.

Capability-based security originates with Dennis and Van Horn (1966), who introduced the concept of capabilities as unforgeable tokens of authority. Miller, Yee, and Shapiro (2003) demonstrated that capability discipline — where authority flows only through explicit delegation — provides confinement properties without ambient authority. EATP's delegation model is a capability system: each Delegation Record is an unforgeable (under the same cryptographic assumptions stated above), attenuatable capability that can only narrow authority. The key difference is that EATP capabilities carry multi-dimensional constraints and audit obligations that classical capability systems do not.

Macaroons (Birgisson et al., 2014) introduced contextual caveats for decentralized authorization — bearer credentials that can be attenuated by adding caveats (conditions) without contacting the issuer. EATP's constraint tightening model is directly analogous: each delegation adds constraints that narrow the operating envelope. Macaroons operate at the request/API level; EATP operates at the organizational governance level, adding human-origin traceability, audit anchors, and reasoning traces that Macaroons do not address.

KeyNote/PolicyMaker (Blaze, Feigenbaum, and Lacy, 1996; Blaze et al., 1999) coined the term "trust management" as a distinct problem: determining whether a set of credentials proves that a request complies with a security policy. EATP addresses the same fundamental problem — does this AI agent's proposed action comply with the trust policy established by human authority? — but extends it with temporal constraint dimensions, graduated verification, and structured reasoning about why decisions were made.

SPKI/SDSI (Ellison et al., 1999) proposed authorization certificates as an alternative to identity certificates, directly binding public keys to authorization rather than to names. EATP's Genesis and Delegation Records serve a similar function: they bind cryptographic keys to specific, constrained authorizations rather than to general identities.

Zanzibar (Pang et al., 2019) provides Google's production-scale authorization system using relationship-based access control, demonstrating that consistent global authorization decisions can be made at scale. EATP's verification operations face the same consistency challenges; Zanzibar's approach to consistent reads and namespace partitioning informs future scaling design for EATP deployments.

Cedar (Cutler et al., 2024) provides a formally verified authorization policy language, with proofs of soundness and decidability. EATP's constraint logic has not been formally verified; Cedar demonstrates both the feasibility and the effort required for formal verification of authorization policy languages. This is acknowledged as a limitation (see Formal Verification, below).

OPA/Rego (Cloud Native Computing Foundation) provides a general-purpose policy engine with a declarative policy language. Many organizations already use OPA for infrastructure authorization. EATP's constraint evaluation could be implemented using OPA as a policy backend; the two systems operate at different layers (OPA evaluates policy rules; EATP manages the trust chains and constraints that produce those rules).

These are mature, well-understood systems and frameworks. The identity and access systems verify identity and permissions. The authorization systems manage delegated authority. The policy systems evaluate compliance rules. The governance frameworks describe what good AI oversight looks like. They answer: "Is this entity who it claims to be?", "Is this entity permitted to perform this operation?", "Does this action comply with policy?", and "What structures should govern AI systems?"

What EATP adds: These existing systems do not verify that an action is within the boundaries of trust that humans established for an AI agent, trace every action back through an unbroken chain to human authority, or record why trust decisions were made. EATP addresses the specific gap between identity/access/authorization infrastructure and accountability-preserving governance for autonomous AI systems. It provides a concrete protocol where governance frameworks provide structural recommendations, and it is designed to operate above (not replace) existing identity and authorization infrastructure.

This is a narrower contribution than inventing the field. It is also a more honest description of what is new here.

Competitive Landscape

Several existing technologies address parts of the trust, authorization, and delegation problem that EATP targets. An honest assessment of how they compare is necessary for anyone evaluating whether EATP adds something new.

Existing Approaches

X.509 / PKI provides hierarchical certificate chains with cryptographic verification. It is the backbone of TLS and code signing. However, X.509 certificates attest to identity, not to behavioral boundaries. A certificate says "this is agent A, issued by authority B." It does not say "agent A may spend up to $10,000 during business hours but may not access PII." X.509 also lacks action-level audit trails and has no concept of constraint tightening across delegation chains.

SPIFFE/SPIRE provides workload identity for distributed systems, with automatic credential rotation and bootstrapping. It solves the "who is this workload?" question but not the "what is this workload authorized to do within these specific boundaries?" question. SPIFFE identities are binary: the workload either has the identity or it does not. There is no mechanism for graduated trust postures or multi-dimensional constraints.

UCAN (User-Controlled Authorization Networks) supports delegated authorization with attenuation, meaning each delegation can only narrow permissions. This is the closest existing technology to EATP's delegation model. UCAN handles capability delegation well but does not include action-level audit trails, multi-dimensional constraint envelopes, or trust postures. UCAN is also designed primarily for decentralized web applications rather than enterprise governance scenarios.

Biscuit provides attenuation tokens with a Datalog-based authorization language. Like UCAN, it supports capability narrowing across delegation chains. Biscuit excels at fine-grained authorization policies but lacks audit trail infrastructure, trust postures, and the human-origin traceability that connects every action back through a chain to a specific human authority.

OAuth 2.0 / OIDC provides delegated authorization for API access with scopes. It is ubiquitous but designed for user-to-application delegation, not agent-to-agent delegation with constraint tightening. OAuth scopes are coarse-grained (e.g., "read:files"), have no built-in mechanism for multi-dimensional constraints, and do not produce action-level audit records.

Contemporary AI Agent Trust Protocols

Four recent papers address the AI agent trust problem directly. EATP must be differentiated from each.

Authenticated Delegation (AD) (South, Marro, Hardjono, Mahari, Whitney, Greenwood, Chan, and Pentland, 2025) proposes a framework for establishing trust in autonomous AI agents by extending OAuth 2.0 and OIDC. AD introduces natural language permission descriptions and delegation chains built on existing identity infrastructure. The key strength is adoption pragmatism: organizations with OAuth deployments can extend existing infrastructure rather than deploying new protocols. AD addresses authorization (what an agent may do) but does not specify multi-dimensional constraint envelopes, graduated trust postures, structured reasoning traces, or action-level audit trails. AD is a position paper describing a framework; no reference implementation is reported. EATP operates at the governance layer above AD's authorization layer: an organization could use AD for runtime OAuth-based authorization while using EATP for governance-level trust establishment, constraint definition, and audit.

Agentic JWT (A-JWT) (Goswami, 2025) extends the JWT standard with an agent identity checksum mechanism for secure delegation in autonomous AI agent systems. A-JWT is positioned for IETF standardization, building on the ubiquitous JWT infrastructure. The protocol addresses token-level trust (is this JWT from a legitimate agent?) but does not specify organizational governance chains, constraint envelopes, trust postures, or reasoning traces. A-JWT secures individual delegation tokens; EATP governs the organizational trust chains that determine what those tokens should contain. The approaches are directly complementary: EATP trust chains can be exported as JWT tokens (and by extension A-JWT tokens) for runtime enforcement.

Omega (Bodea et al., 2025) provides hardware-rooted trust for AI agents through Confidential Virtual Machines (AMD SEV-SNP) and Confidential GPUs (NVIDIA H100). Omega establishes nested isolation for multi-agent workloads, differential attestation of agent code, models, and LoRA weights, and a declarative policy language enforced at a privileged memory level inaccessible to agent code. The attestation protocol is formally verified using Tamarin Prover. This addresses a threat model that EATP does not: compromised execution environments. However, Omega treats trust as binary (attested or untrusted), provides only a single human-ap-

proval predicate, and does not specify organizational governance chains, multi-dimensional constraint envelopes, trust postures, or reasoning traces. The approaches are complementary at different layers: Omega secures the execution environment (hardware trust), EATP governs what the agent within that environment is authorized to do (organizational trust). EATP audit anchors could be stored in Omega's tamper-evident log for defense in depth.

OpenPort Protocol (OPP) (Zhu et al., 2026) specifies a governance-first protocol for securing AI agent access to application tools through a server-side gateway. OPP introduces preflight impact binding — a cryptographic hash over a canonicalized impact summary (per RFC 8785) that pins execution to a pre-computed effect — and a State Witness profile that addresses time-of-check/time-of-use vulnerabilities in delayed-approval flows. Auto-execution is treated as an explicit, time-bounded, revocable capability rather than a default. OPP operates at the gateway layer, governing how agent actions pass through tool access boundaries. It does not address human-origin traceability, inter-agent delegation chains, or structured reasoning traces. The two protocols address different points in the agent action lifecycle and are naturally composable: EATP establishes trust and constraints upstream, OPP governs the gateway through which trusted agents operate. An EATP Constraint Envelope could be consumed by an OPP gateway to derive its authorization policy.

EATP's Distinguishing Contributions

Three protocol features are unique to EATP among the systems surveyed above:

Structured reasoning traces with cryptographic binding. No surveyed system records WHY trust decisions were made. EATP reasoning traces provide structured rationale (decision, alternatives considered, evidence, methodology, confidence score) with dual-binding cryptographic signing (reasoning hash in parent signature prevents post-hoc substitution; independent reasoning signature enables content verification), five-level enterprise confidentiality classification (PUBLIC through TOP_SECRET), and SD-JWT selective disclosure for privacy-preserving audit. This addresses the gap between "what happened" (which audit logs answer) and "why it happened" (which no existing protocol answers).

Trust postures with graduated autonomy. No surveyed system formalizes the relationship between trust level and human involvement. EATP's five named postures (Pseudo-Agent through Delegated) define specific human roles at each autonomy level, with explicit upgrade and downgrade mechanics. This maps directly to organizational practice — the same agent may operate under different postures depending on demonstrated performance, task sensitivity, or changed conditions.

Verification gradient with four outcome categories. No surveyed system provides graduated verification outcomes. Binary pass/fail verification forces a choice between blocking legitimate actions and permitting questionable ones. EATP's four outcomes (Auto-approved, Flagged, Held, Blocked) focus human attention on boundary cases rather than distributing it across all actions.

What No Single System Provides

Each of the systems surveyed above handles aspects of the trust, authorization, and governance problem. The following table summarizes feature coverage across infrastructure protocols, contemporary AI agent trust protocols, and EATP:

Requirement	X.509	SPIFFE	UCAN	Biscuit	OAuth	AD	A-JWT	Omega	OPP	EATP
Hierarchical trust chains	Yes	Yes	Yes	Yes	Partial	Yes	Yes	No	No	Yes
Constraint tightening	No	No	Yes	Yes	No	No	No	No	No	Yes
Multi-dimensional constraints	No	No	No	Partial	No	No	No	No	No	Yes
Action-level audit trail	No	No	No	No	No	No	No	Yes	Yes	Yes
Trust postures (graduated autonomy)	No	No	No	No	No	No	No	No	No	Yes
Human-origin traceability	No	No	No	No	No	Yes	Partial	No	No	Yes
Decision reasoning traces	No	No	No	No	No	No	No	No	No	Yes
Hardware trust root	No	No	No	No	No	No	No	Yes	No	No
Preflight impact binding	No	No	No	No	No	No	No	No	Yes	No
Formal verification	No	No	No	Partial	No	No	No	Yes	No	No
Interop with existing formats	N/A	X.509	JWT	N/A	JWT	OAuth	JWT	N/A	N/A	JWT, W3C VC, DID, UCAN, SD-JWT, Biscuit

This is not a claim that EATP is superior to these systems. They operate at different layers and address different aspects of the trust problem. X.509, SPIFFE, and OAuth solve identity and access. UCAN and Biscuit solve delegated authorization. AD and A-JWT extend existing authorization infrastructure for AI agents. Omega secures the execution environment with hardware trust. OPP governs agent interactions at organizational boundaries. EATP solves accountability-preserving governance — establishing who authorized what, within what multi-dimensional constraints, and why — and is designed to complement these systems rather than replace them. EATP trust chains can be exported to JWT, UCAN, W3C VC, and DID formats precisely because the protocol is intended to integrate with existing infrastructure.

NIST's AI Risk Management Framework (2023) recommends governance structures that include authorization chains, risk boundaries, monitoring, and audit records. No single existing protocol implements this full set. EATP is one attempt to do so. Whether it succeeds is an empirical question; the comparison above describes the structural gap it targets.

Ecosystem Context

EATP is one component of a governance architecture. Understanding its relationship to the companion frameworks clarifies what it does and does not address.

The CARE / EATP / CO Triangle

The Terrene Foundation publishes three peer specifications:

CARE (Collaborative Autonomous Reflective Enterprise) (Hong, 2026a) defines the philosophical framework: what is the human for in an AI-augmented organization? CARE's answer is that humans provide trust, judgment, and accountability — the things AI cannot provide for itself. CARE establishes the "why."

EATP (Enterprise Agent Trust Protocol) operationalizes CARE's philosophy into a verifiable protocol. EATP's trust chains, constraint envelopes, and audit anchors are the cryptographic implementation of CARE's principle that trust originates in human commitment and must remain traceable. EATP establishes the "how" for trust infrastructure.

CO (Cognitive Orchestration) (Hong, 2026c) defines how humans structure AI's work. CO provides a five-layer architecture for maintaining institutional context, guardrails, and operating procedures across human-AI collaboration. COC (Cognitive Orchestration for Codegen) (Hong, 2026d) is the initial domain application, instantiating CO for software development. CO establishes the "how" for operational methodology.

Each specification is independently useful. An organization can adopt EATP without CARE or CO. It can adopt CARE's philosophy without implementing EATP's protocol. It can use CO's methodology without either. But together, they form a coherent governance stack: CARE defines the principles, EATP provides verifiable trust lineage, and CO structures the work. The Constrained Organization thesis (Hong, 2026e) examines the organizational form that emerges when all three are deployed together.

Independent Adoptability

EATP is designed for adoption without requiring the rest of the Terrene Foundation's architecture. The reference implementation ships as the `kailash.trust` namespace of kailash-py (`pip install kailash`); using EATP requires only that namespace and carries no dependency on CARE, CO, or the broader Kailash orchestration layers (DataFlow, Nexus, Kaizen). An organization can integrate EATP trust verification into any existing system — a FastAPI service, a Django application, an MCP server, an A2A agent network — using only the trust namespace.

This design is intentional. Trust infrastructure creates value through adoption breadth. Making adoption contingent on a larger framework would restrict that breadth. The trust namespace is usable on its own to lower the barrier to adoption, not to create an on-ramp to a larger platform.

Open-Source Reference Implementation

kailash-py is the open-source reference implementation, published under Apache 2.0 by the Terrene Foundation. EATP lives in its `kailash.trust` namespace; the broader platform (Core SDK, DataFlow, Nexus, Kaizen) layers orchestration, registry, and multi-agent coordination on top of those same trust primitives. Organizations that want only trust infrastructure depend on the trust namespace; those that want full orchestration adopt the broader platform. All of it is Apache 2.0, and none of it creates lock-in.

Reference Implementation

The reference implementation lives in the `kailash.trust` namespace of **kailash-py v2.28.1** (`pip install kailash`), the Foundation-owned Apache 2.0 Python SDK, and implements the complete protocol specification. The trust namespace spans 222 modules (~101,000 lines of source) with 209 test files; it houses both the EATP protocol layer and the PACT governance engine, which builds on the same trust primitives.

Core Protocol Implementation

Four operations: ESTABLISH creates genesis records and binds agent keys to authority. DELEGATE transfers capabilities with monotonic constraint tightening. VERIFY validates trust chains at three levels of thoroughness (QUICK, STANDARD, FULL). AUDIT records actions in a hash-linked, append-only trail.

Five constraint dimensions: Financial, Operational, Temporal, Data Access, and Communication, with six built-in constraint templates (governance, finance, community, standards, audit, minimal) and a template customization API.

Structured reasoning traces: Optional reasoning traces with five-level confidentiality classification (PUBLIC through TOP_SECRET) can be attached to Delegation Records and Audit Anchors. Reasoning traces use a dual-binding signing model: the reasoning trace hash is bound into the parent record's signature (preventing post-hoc substitution), while a separate reasoning signature enables independent content verification. SD-JWT-based selective disclosure supports confidentiality-driven redaction. A `REASONING_REQUIRED` constraint type enables organizations to mandate reasoning documentation for compliance workflows, with enforcement escalating through the verification gradient (advisory at STANDARD, mandatory at FULL).

Enforcement and Operations

Enforcement: StrictEnforcer for production use (blocks unauthorized actions, holds for human review) and ShadowEnforcer for observation mode (logs what would be blocked without interrupting). A challenge-response mechanism enables runtime verification of agent capabilities. Python decorators provide three-line integration for any async function.

Messaging: Inter-agent messaging with cryptographic signing, verification, and replay protection. Agents exchange messages through typed channels with envelope authentication.

Cascade revocation: Push-based broadcast revocation with automatic downstream propagation. When trust is revoked at any level, all agents holding delegated authority from that source are notified and invalidated.

Key rotation: Cryptographic key rotation with automatic re-signing of active records, enabling key lifecycle management without trust chain interruption.

Multi-signature: Support for multi-party signing requirements on delegation records, enabling organizational policies that require multiple human authorities to approve high-stakes delegations.

Circuit breaker: Fault tolerance mechanism that automatically downgrades trust postures or suspends agent operations when error rates exceed configurable thresholds.

Storage and Audit

Storage backends: In-memory (testing and development), filesystem (single-machine deployment), and PostgreSQL (production) — all within the `kailash.trust` namespace. The storage interface is abstract; additional backends can be implemented against the `TrustStore` protocol.

Append-only audit store: PostgreSQL-backed audit storage with integrity verification, providing a persistent and tamper-evident record of all trust operations.

Merkle tree audit verification: O(log n) proof generation for efficient verification of audit chain integrity, complementing the linear hash chain with a tree structure that enables selective verification without full chain traversal.

Canonical conformance vectors: A published test-vector corpus (`audit-chain-canonical.json`, `trace-event-canonical.json`) pins the byte-level canonical form used for audit-anchor and reasoning-trace hashing — including Unicode and JSON-canonicalization edge cases per RFC 8259 — so that independent implementations can verify conformance against the reference rather than relying on prose alone.

Trust scoring: Quantitative trust score computation based on delegation chain properties, constraint compliance history, and operational behavior patterns.

Interoperability and Integration

Six credential formats: Trust chains can be exported to and imported from JWT (RFC 7519), W3C Verifiable Credentials 2.0, Decentralized Identifiers (DID), UCAN v0.10.0, SD-JWT (selective disclosure), and Biscuit-inspired attenuation tokens. W3C VC and DID export enables cross-organizational trust exchange.

Google A2A protocol integration: Automatic EATP capability card generation for A2A agent networks, enabling cross-agent trust establishment in multi-agent systems that use Google's Agent-to-Agent protocol.

MCP server: An MCP (Model Context Protocol) server exposing 5 trust tools and 4 resource templates. AI agents using MCP-compatible frameworks can establish trust, verify capabilities, and record audit entries through standard MCP tool calls.

Enterprise System Agent (ESA) framework: Trust-aware proxy agents for database systems (PostgreSQL, MySQL, SQLite), with automatic capability-based access control. ESAs wrap existing data infrastructure with EATP constraint enforcement without modifying the underlying systems.

Agent registry: PostgreSQL-backed registry for agent metadata, capability declarations, and trust chain status — enabling discovery of trusted agents within an organizational network.

CLI: A trust-plane CLI (`attest`) provides lifecycle management — establishing and delegating authority, verifying chains, approving or denying held actions, applying constraint templates, and exporting verification bundles.

Governance Engine

A policy engine, rate limiter, and cost estimator enable organizational governance policies to be expressed as machine-enforceable rules that operate above individual constraint envelopes — supporting organization-wide budgets, rate limits, and cost projections across all active trust chains.

CARE Platform Demonstrator

The CARE Platform (11,334 lines of source code across 55 modules, with 20,472 lines of test code containing 1,304 automated tests) exercises all EATP operations in a multi-agent team configuration. A five-agent decision-making team uses EATP trust chains for authority delegation, constraint enforcement, and audit trail generation. The demonstrator illustrates that the protocol supports real multi-agent coordination, not merely isolated trust operations.

What the SDK Does Not Include

Enterprise-scale key management (HSM integration) and formal protocol verification (ProVerif, Tamarin) are not part of the current SDK. Cross-organizational federation governance — the bilateral agreements and trust resolution policies that organizations need when establishing mutual trust — is deferred to future work, though the protocol's W3C VC, DID, and A2A integration provides the interoperability substrate on which federation can be built.

What This Does Not Solve

Any framework that hides its limitations is not worth trusting. EATP has fundamental ones.

Adversary Non-Goals

EATP does not defend against: a compromised human principal who intentionally creates malicious delegations; constraint gaming (achieving prohibited outcomes through sequences of individually permitted actions); compromised trust stores (integrity of the trust store is assumed); side-channel attacks on cryptographic implementations; multi-agent collusion — two or more compromised agents coordinating to circumvent constraints that hold against any single compromised agent (note: a compromised non-leaf agent with delegation authority can create new delegations to agents under adversary control, effectively producing a multi-agent compromise from a single-agent breach; the blast radius of non-leaf compromise is therefore larger than the single-agent model implies); and denial-of-service attacks against the trust store or verification infrastructure (e.g., flooding with invalid delegation requests or triggering excessive revocation notifications).

Constraint Gaming

This is the most important limitation. Agents might achieve prohibited outcomes through sequences of individually permitted actions.

An agent cannot transfer funds directly, but it can approve fraudulent invoices, achieving fund transfer through the accounts payable process. An agent cannot access personal data directly, but it can generate aggregate reports with filters so narrow that individuals are identifiable. An agent cannot terminate contracts, but it can modify terms until they become so unfavorable that the counterparty terminates.

This is not a bug. It is a fundamental limitation of boundary-based governance. A constraint system that could fully prevent gaming would require complete specification of all undesirable outcomes, reasoning about causal chains from permitted actions to outcomes, and prediction of adversarial agent behavior. This is equivalent to the alignment problem in AI safety. EATP does not solve it.

What EATP provides: audit trails for detection, cumulative constraints to limit repeated exploitation, and human-on-the-loop monitoring for emergent patterns. What organizations must provide: active monitoring for outcome-based anomalies, incident investigation, constraint refinement based on detected patterns, and recognition that this is ongoing work with no final solution.

Compromised Genesis Authority

If the human at the root of trust is compromised, coerced, bribed, negligent, or simply wrong, the entire chain inherits that compromise. EATP authenticates chains; it cannot evaluate the wisdom of those who create them. This is inherent to hierarchical trust models.

More specifically, if the authority's signing key is compromised, all downstream chains are invalidated — the attacker can create arbitrary delegations that appear legitimate. Partial mitigations include multi-signature genesis (requiring two or more principals to agree), time-delay mechanisms (allowing a review period before authority takes effect), and independent witness cosigning. These are operational practices layered on top of the protocol, not protocol features.

Correct but Unwise Constraints

EATP verifies that constraints are respected. It does not verify that constraints were wisely set. An organization that defines overly permissive constraints will have agents that operate within those constraints, and may still cause harm. The constraints are only as good as the human judgment that created them.

Implementation Vulnerabilities

The protocol's security depends on correct implementation. Bugs in verification code, key management, or cryptographic operations create vulnerabilities that no protocol design can prevent. The reference SDK has not undergone independent security audit.

Key Revocation and Rotation

The specification does not define a key rotation protocol. The reference implementation supports key expiration via temporal constraints, but the protocol does not define a key revocation mechanism (e.g., CRL or OCSP equivalent). If a delegator's key is compromised after signing a delegation, that delegation remains valid until expiry. In production, a compromised agent key should trigger cascade revocation of all delegations rooted in that key. The SDK implements cascade revocation operationally, but this mechanism is not formalized in the protocol specification.

Social Engineering

Humans in the trust chain can be deceived into creating delegations or constraints they would not otherwise create. EATP governs the technical chain; it cannot govern the humans within it.

Cross-Organizational Trust

The SDK provides interoperability mechanisms for cross-organizational trust exchange: W3C Verifiable Credentials for portable trust chain representation, Decentralized Identifiers for cross-domain agent identity, and Google A2A protocol integration for capability card exchange between agent networks. These formats enable Organization A to export an EATP trust chain as a W3C VC that Organization B can verify independently.

What remains unspecified is the federation governance layer — the bilateral agreements, trust resolution policies, and key discovery protocols that organizations need when establishing mutual trust at scale. The interoperability substrate exists; the governance framework for operating it across organizational boundaries is deferred to future work.

Cross-organizational trust introduces specific challenges for reasoning traces. When trust chains cross organizational boundaries, conflicting confidentiality policies may apply: Organization A may classify reasoning as `confidential` while Organization B's compliance requires disclosure. Three resolution approaches are anticipated: (1) highest-classification-wins, where the reasoning trace is classified at the higher of the two levels; (2) originator-controls, where the issuing organization's classification governs; (3) bilateral agreement, where organizations negotiate classification handling as part of their federation arrangement. The protocol's design principle — confidentiality can only be tightened at import, never relaxed — is consistent with the constraint tightening principle governing delegation chains. A full cross-organizational confidentiality negotiation protocol is deferred to future work.

Reasoning Trace Integrity

Reasoning traces document the stated rationale for a decision, not the quality of the decision itself. An agent can document flawless reasoning and still make the wrong call. Post-hoc rationalization — writing a convincing justification after the decision is already made — is a real risk that structural verification can mitigate but not eliminate.

Reasoning traces use a dual-binding signing model: the `reasoning_trace_hash` is included in the parent record's signing payload, binding the reasoning to the delegation at creation time. Replacing a reasoning trace invalidates the parent signature, preventing same-signer substitution. However, an authorized signer can always create a *new* delegation record with different reasoning — this is a property of any system where authorized parties issue records, not a protocol vulnerability. Organizations that need to detect unauthorized re-delegation should implement append-only storage and monitor for duplicate delegations with different reasoning.

The `REASONING_REQUIRED` constraint mandates presence, not quality. A minimal reasoning trace (e.g., a one-word `decision` and `rationale`) satisfies the protocol-level check. Meaningful quality enforcement — minimum field lengths, non-empty evidence, substantive rationale — is a deployment responsibility. Organizations using `REASONING_REQUIRED` for regulatory compliance should implement content quality validation above the protocol layer.

The confidentiality classification system (PUBLIC through TOP_SECRET) is advisory classification enforced by cooperating systems, not cryptographic enforcement at the protocol level. SECRET and TOP_SECRET require encryption at rest — a deployment responsibility, not a protocol guarantee. And in litigation, detailed reasoning traces are discoverable records; organizations must weigh the compliance benefit against the legal exposure of having that reasoning available to opposing counsel.

Formal Verification

The protocol's constraint logic has not been formally verified. While the design is straightforward (monotonic constraint tightening, hash-linked audit chains), formal proofs of correctness have not been produced. Production deployments in safety-critical domains should commission independent formal analysis.

Why Open Standards and Open Source Matter

This is where the institutional argument becomes economic.

Enterprise AI trust infrastructure cannot be proprietary. If the protocol that governs AI accountability is owned by a single vendor, every organization using it depends on that vendor's continued existence, goodwill, and commercial decisions. That is not trust infrastructure. That is vendor dependency dressed in governance language.

The EATP specification is published under CC BY 4.0. Anyone can read it, implement it, extend it, or build competing implementations. The reference implementation is Apache 2.0. Anyone can use it, modify it, or build commercial products on it. The Apache 2.0 license file is in the project repository (https://github.com/terrene-foundation/) - perpetual, irrevocable, and it includes an automatic patent grant under Section 3.

Why this structure rather than pure proprietary? Because the economics of trust infrastructure are different from the economics of application software. Trust infrastructure creates value through adoption breadth. A trust protocol used by one organization is marginally useful. A trust protocol used across an industry is critical infrastructure. Proprietary ownership restricts adoption. Open standards accelerate it.

Nothing beats having the Apache 2.0 license in the repository. It is the most concrete trust signal available: not a promise, not a governance structure, not a future commitment - a legal instrument already in effect.

Regulatory Alignment

EATP is designed to support, not guarantee, compliance with emerging AI governance frameworks.

EU AI Act

Requirement	Article	EATP Support
Human oversight	Art. 14	Trust postures enable graduated human involvement
Risk management	Art. 9	Constraint envelopes encode risk boundaries
Record-keeping	Art. 12	Audit anchors maintain complete action records
Transparency	Art. 13	Trust chains make authorization visible
Accountability	Art. 26	Delegation chains identify responsible humans

These articles apply to "high-risk AI systems" as defined in the regulation. Whether specific deployments qualify depends on use case. Organizations should obtain independent legal assessment.

NIST AI Risk Management Framework

At a high level, EATP maps to the four NIST AI RMF functions: Govern (genesis records establish organizational governance), Map (capability attestations describe agent functions), Measure (audit trails provide performance evidence), and Manage (constraint envelopes implement risk controls). The mapping is suggestive rather than precise; the NIST functions are broader than any single protocol operation.

The NIST framework's emphasis on documented authorization chains, risk boundaries, and auditable decision records aligns structurally with EATP's five elements. EATP provides one concrete protocol implementation of the governance approach NIST recommends.

Industry Standards

EATP is designed to complement SOC 2 audit requirements, ISO 27001 information security controls, and GDPR data processing accountability obligations.

The Traceability Distinction

One distinction must be made with absolute clarity, because getting it wrong would undermine the entire thesis.

EATP provides traceability, not accountability.

Traceability is the ability to trace any AI action back through a chain of delegations to human authority. EATP delivers this.

Accountability requires that humans understand what the AI did and why, evaluate whether the action was appropriate given context, and bear consequences for outcomes of AI actions they authorized. EATP does not deliver this. No protocol can.

Traceability is necessary for accountability but not sufficient. It provides the evidentiary foundation: who authorized what, within what constraints, with what outcome. Whether organizations build the practices that convert traceability into actual accountability is an organizational question, not a technical one.

EATP provides the infrastructure. Accountability requires that humans use it.

Security Recommendations

For organizations evaluating or implementing EATP:

Cryptographic minimums: SHA-256 for hashing. Ed25519 for signatures. HSM (Hardware Security Module) storage for genesis and delegation signing keys. The specification and reference SDK use SHA-256 and Ed25519 exclusively. Implementations may substitute SHA-3 or ECDSA P-256 if required by organizational policy, but interoperability with the reference SDK requires SHA-256 and Ed25519.

The reference SDK uses Ed25519 via PyNaCl for all signing and verification operations. Key generation, serialization, and management are provided. Production deployments should evaluate whether the default key storage (filesystem or PostgreSQL) meets their security requirements or whether HSM integration is necessary.

Production implementations should undergo independent security review (Anderson, 2020). The protocol specification describes the architecture; it does not validate specific implementations.

Alternatives Considered

Intellectual honesty requires acknowledging what else was considered and why it was set aside.

Behavior-based trust - trust agents based on observed behavior rather than pre-established chains; cannot establish initial accountability and is gameable.

Consortium-based verification - multiple independent parties verify each action; adds latency and coordination complexity that may exceed the security benefit for enterprise use cases.

Outcome-based accountability - judge actions by results rather than authorization; is retrospective only and creates perverse incentives.

Insurance-based governance - let insurers price AI risk and set requirements; requires AI liability markets that do not yet exist.

Regulatory mandate - wait for regulators to define required governance; means operating without governance while regulation lags technology.

I may have underweighted federated trust models, capability-based security, continuous learning constraints, and decentralized autonomous governance. These alternatives deserve continued investigation by the broader community.

Why This Matters

The AI era is not asking organizations to do the same things faster. It is asking them to do fundamentally different things. The question is not “how do we automate our processes?” It is “how do we build institutional structures that make AI-driven operations trustworthy, accountable, and economically valuable?”

EATP addresses one piece of that question: the trust infrastructure that connects autonomous AI action back to human authority. It is not the complete answer. It is the evidentiary foundation without which the rest of the answer cannot be built.

The accountability gap is real. Organizations are filling it with workarounds - approval queues, review committees, restrictive policies that defeat the purpose of AI adoption. These workarounds will not scale. At some point, organizations must choose between meaningful AI autonomy and meaningful human accountability. EATP’s thesis is that this is a false choice. You can have both, if you build the right infrastructure.

Whether that thesis holds is an empirical question. The thesis fails if constraint-based verification proves insufficient — if organizations consistently find that the gap between constraint compliance and actual accountability cannot be bridged by operational practices. It also fails if the overhead of trust chain management (key management, constraint engineering, audit storage) exceeds the governance value it provides. That commitment to testability is what separates a framework from a sales pitch.

Relationship to Kailash

EATP defines the open standard. kailash-py (v2.28.1, the `kailash.trust` namespace) is the open-source reference implementation, published under Apache 2.0 by the Terrene Foundation.

The stack is designed so that small teams with deep domain expertise can build enterprise-quality AI products - a consequence of the broader insight that the AI era fundamentally changes the economics of enterprise software creation. Pre-constructed building blocks, open standards, and AI-assisted development mean that what previously required enterprise budgets is now accessible to organizations of any size.

Two patent applications are pending (PCT/SG2024/050503 and P251088SG; favorable IPRP received, national phase filings in progress in Singapore and the United States; no patent has been granted). Under Apache 2.0 Section 3, all users of the reference implementation receive a perpetual, royalty-free patent license with defensive termination if the user initiates patent litigation. The author commits that if the Apache 2.0 patent grant in Section 3 of the Apache License is ever narrowed, revoked, or supplemented with additional restrictions, all patent claims in these applications shall be irrevocably dedicated to the public domain.

EATP's value as a standard does not depend on any single implementation.

References

Foundational Systems

1. Dennis, J. B., & Van Horn, E. C. (1966). Programming Semantics for Multiprogrammed Computations. *Communications of the ACM*, 9(3), 143-155.
2. Blaze, M., Feigenbaum, J., & Lacy, J. (1996). Decentralized Trust Management. *Proceedings of the 1996 IEEE Symposium on Security and Privacy*, 164-173.
3. Blaze, M., Feigenbaum, J., Ioannidis, J., & Keromytis, A. D. (1999). The KeyNote Trust-Management System, Version 2. IETF RFC 2704.
4. Ellison, C., Frantz, B., Lampson, B., Rivest, R., Thomas, B., & Ylonen, T. (1999). SPKI Certificate Theory. IETF RFC 2693.
5. Haber, S., & Stornetta, W. S. (1991). How to Time-Stamp a Digital Document. *Journal of Cryptology*, 3(2), 99-111.
6. Merkle, R. C. (1980). Protocols for Public Key Cryptosystems. *Proceedings of the IEEE Symposium on Security and Privacy*, 122-134.
7. Miller, M. S., Yee, K.-P., & Shapiro, J. (2003). Capability Myths Demolished. Technical Report SRL2003-02, Systems Research Laboratory, Johns Hopkins University.
8. Birgisson, A., Politz, J. G., Erlingsson, U., Taly, A., Vrable, M., & Lentczner, M. (2014). Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud. *Proceedings of the 2014 Network and Distributed System Security Symposium (NDSS)*.
9. Pang, R., Caceres, R., Burrows, M., Chen, Z., Dave, P., Germer, N., Golynski, A., Graney, K., Kang, N., Kissner, L., Korn, J. L., Parmar, A., Richards, C. D., & Wang, M. (2019). Zanzibar: Google's Consistent, Global Authorization System. *Proceedings of the USENIX Annual Technical Conference (ATC)*, 33-46.

10. Cutler, J. W., Disselkoen, C., Eline, A., He, S., Headley, K., Hicks, M., Hietala, K., Ioannidis, E., Kastner, J., Mamat, A., McAdams, D., McCutchen, M., Rungta, N., Torlak, E., & Wells, A. M. (2024). Cedar: A New Language for Expressive, Fast, Safe, and Analyzable Authorization. *Proceedings of the ACM on Programming Languages*, 8(OOPSLA1). Also *arXiv:2403.04651*.

Infrastructure Standards

11. IETF RFC 6749. The OAuth 2.0 Authorization Framework.
12. SPIFFE. Secure Production Identity Framework for Everyone. https://spiffe.io
13. UCAN Working Group. (2023). UCAN Specification v0.10.0. https://ucan.xyz
14. Biscuit. (2023). Biscuit Authorization Token Specification. https://www.biscuitsec.org
15. Cloud Native Computing Foundation. Open Policy Agent (OPA). https://www.openpolicyagent.org

Contemporary AI Agent Trust Protocols

16. South, T., Marro, S., Hardjono, T., Mahari, R., Whitney, C. D., Greenwood, D., Chan, A., & Pentland, A. (2025). Authenticated Delegation and Authorized AI Agents. *arXiv:2501.09674*.
17. Goswami, A. (2025). Agentic JWT: A Secure Delegation Protocol for Autonomous AI Agents. *arXiv:2509.13597*.
18. Bodea, T., Misono, M., Pritzi, J., Sabanic, P., Sommer, T., Unnibhavi, H., Schall, D., Santos, N., Stavrakakis, D., & Bhatotia, P. (2025). Trusted AI Agents in the Cloud. *arXiv:2512.05951*.
19. Zhu, G., Wang, C., Wang, Z., Li, Z., & Li, Q. (2026). OpenPort Protocol: A Security Governance Specification for AI Agent Tool Access. *arXiv:2602.20196*.

Governance Frameworks

20. European Union. (2024). Regulation (EU) 2024/1689 (AI Act).
21. National Institute of Standards and Technology. (2023). *AI Risk Management Framework (AI RMF 1.0)*.
22. Irving, G., Christiano, P., & Amodei, D. (2018). AI Safety via Debate. *arXiv:1805.00899*.
23. Anderson, R. (2020). *Security Engineering*. 3rd Edition. Wiley.

Companion Papers

24. Hong, J. (2026a). "CARE: A Core Thesis." White Paper Series, Version 2.1. Terrene Foundation. https://github.com/terrene-foundation/publications/blob/main/CARE-Core-Thesis.pdf. Companion paper providing the philosophical governance framework that EATP operationalizes.
25. Hong, J. (2026c). "CO: A Core Thesis." White Paper Series, Version 1.1. Terrene Foundation. https://github.com/terrene-foundation/publications/blob/main/CO-Core-Thesis.pdf. Companion paper defining the domain-agnostic methodology for structuring human-AI collaboration.
26. Hong, J. (2026d). "COC: A Core Thesis." White Paper Series, Version 1.1. Terrene Foundation. https://github.com/terrene-foundation/publications/blob/main/COC-Core-Thesis.pdf. Domain application of CO for software development.

27. Hong, J. (2026e). "The Constrained Organization: An Organizational Model for Enterprise AI Governance." White Paper Series, Version 1.0. Terrene Foundation. https://github.com/terrene-foundation/publications/blob/main/Constrained-Organization-Thesis.pdf. Companion paper on organizational design for AI governance stewardship.

A Note on How This Paper Was Produced

This paper was written using the process it describes.

The initial draft was generated by a team of AI agents:

- **research analysts** that explored the knowledge base and source repositories, wrote structured drafts, and cross-referenced claims against anchor documents.
- A separate **alignment-critic agent** red-teamed the output, identifying eight issues including factual errors, overstatements, and inconsistencies with source material.
- A **debate-expert agent** challenged the draft's arguments and flagged unverifiable claims.

The author directed every iteration. Instructions included:

- remove claims that cannot be verified
- strip out references to structures that do not yet exist
- simplify the disclosure to plain facts
- explain technical terms for non-technical readers
- ensure the paper reads as a standards proposition rather than marketing

The author made direct edits to tone, phrasing, and substance throughout. Multiple rounds of revision followed, each narrowing the gap between what the agents produced and what the author judged to be honest and useful.

The AI agents drafted. The author defined the boundaries, evaluated the output, caught what the agents missed, and decided what stayed. That is human-on-the-loop in practice, not a theoretical model, but the process that produced this document.

The tools used were Claude Code (Anthropic) with specialized subagents for research, alignment checking, and adversarial review. The full conversation history, including every instruction, every correction, and every piece of content the author rejected, is retained in the project repository.

See also: Hong, J. (2026a). "CARE: A Core Thesis" for the governance framework that EATP operationalizes. Hong, J. (2026c). "CO: A Core Thesis" for domain-agnostic human-AI collaboration methodology. Hong, J. (2026d). "COC: A Core Thesis" for development methodology. Hong, J. (2026e). "The Constrained Organization" for organizational design.

Quick Reference

Five Elements of the Trust Lineage Chain

1. **Genesis Record** — Organizational root of trust; human executive commits to AI governance accountability
2. **Delegation Record** — Authority transfer with constraint tightening; delegations can only narrow, never expand. Optional reasoning trace documents WHY the delegation was made.
3. **Constraint Envelope** — Five-dimensional operating boundaries (Financial, Operational, Temporal, Data Access, Communication)
4. **Capability Attestation** — Signed declaration of authorized agent capabilities; prevents capability drift
5. **Audit Anchor** — Tamper-evident execution record; each anchor hashes the previous, forming a verifiable chain. Optional reasoning trace documents WHY the agent chose this action.

Verification Gradient

- **Auto-approved** — Within all constraints; execute and log
- **Flagged** — Near boundary; execute and highlight for review
- **Held** — Soft limit exceeded; queue for human approval
- **Blocked** — Hard limit violated; reject with explanation

Five Trust Postures

1. **Pseudo-Agent** — No autonomy; human in-the-loop
2. **Supervised** — Low autonomy; human in-the-loop
3. **Shared Planning** — Medium autonomy; human on-the-loop
4. **Continuous Insight** — High autonomy; human on-the-loop
5. **Delegated** — Full autonomy; human over-the-loop (remote monitoring)

Critical Distinctions

- **Traceability is not accountability**: EATP provides traceability; accountability requires organizational practices
- **Reasoning traces are rationale, not proof**: They document stated justification, not decision quality; post-hoc rationalization remains a risk
- **Constraint gaming cannot be prevented**: It can only be detected, traced, and mitigated through ongoing human oversight
- **Performance figures are design targets**: Not benchmarks; actual performance depends on implementation